

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Analytical Problem Solving
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/nonrestoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	Kolar Umme Sadiya	Reg. No.:	192425117
Section / Batch:	3 rd year/AI&ML	Date:	03/08/2026

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A computer system uses 8-bit 2's complement representation to process signed integer arithmetic. Consider two numbers: -52 and $+19$. Analyze in detail how the processor performs both addition and subtraction, including binary conversion, sign extension, intermediate steps, and the final result. Determine whether overflow occurs in each operation and explain how the processor detects and handles overflow.
2. A modern processor is designed with an 8-bit carry look-ahead adder to accelerate arithmetic operations. Given two binary operands, analyze how the generate and propagate signals are produced, how carry bits are calculated simultaneously, and compare the execution time of both adders. Justify why CLA is preferred in high-performance processors.
3. A digital processor performs signed multiplication using Booth's algorithm for efficient arithmetic computation. Multiply $+11$ and -7 , and analyze each iteration of the algorithm by showing the contents of the accumulator, multiplier register, Booth bit, and the arithmetic right shifts. Explain how Booth's algorithm minimizes the number of addition and subtraction operations while correctly handling signed operands.
4. A binary arithmetic unit supports both restoring and non-restoring division techniques for integer division. Analyze the execution of the division of $25 \div 4$ using both methods by showing the intermediate remainder values, quotient generation, and register contents after each step. Compare the algorithms based on execution time, hardware complexity, and computational efficiency, and explain why one method is preferred over the other.
5. A floating-point unit (FPU) processes real numbers according to the IEEE 754 standard. Analyze the representation of the decimal number $+26.75$ in both single-precision (32-bit) and double-precision (64-bit) formats by identifying the sign, exponent, and mantissa fields. Further, analyze the floating-point addition of 26.75 and -8.5 , explaining exponent alignment, mantissa operations, normalization, rounding, and the effect of precision on the final result.

Code of Conduct

I Kolar Umme Sadiya 192425117(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kolar Umme sadiya

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

1. 8-bit 2's Complement Arithmetic

Given:

- $A = -52$
- $B = +19$

Step 1: Binary Conversion

$$+52 = 00110100$$

-52 (2's complement):

$$00110100$$

$$11001011 \text{ (1's complement)}$$

$$11001100 \text{ (+1)}$$

So,

$$-52 = 11001100$$

$$+19 = 00010011$$

(A) Addition ($-52 + 19$)

$$11001100$$

$$+00010011$$

$$11011111$$

$$\text{Result} = 11011111$$

Find magnitude:

$$11011111$$

$$00100000$$

$$00100001$$

$$= 33$$

$$\text{Final Result} = -33$$

Overflow: No

Reason: Overflow occurs only when two numbers with the same sign are added and the result has a different sign. Here one number is positive and the other is negative.

(B) Subtraction ($-52 - 19$)

Convert $+19$ into -19 :

$$00010011$$

11101100

11101101

Now add:

11001100

+11101101

10111001

Ignore the carry.

Find magnitude:

10111001

01000110

01000111

= 71

Final Result = -71

Overflow: No

Since -71 lies within the 8-bit signed range (-128 to +127).

Overflow Detection

The processor detects overflow by comparing the carry into the most significant bit (MSB) and the carry out of the MSB.

- Carry into MSB = Carry out of MSB → No Overflow
- Carry into MSB $\neq$ Carry out of MSB → Overflow

2. Carry Look-Ahead Adder (CLA)

An 8-bit Carry Look-Ahead Adder speeds up addition by generating carry bits simultaneously.

Assume:

A = 10110110

B = 01100101

Generate Signal

$G_i = A_i \times B_i$

Generate becomes 1 when both input bits are 1.

Propagate Signal

$P_i = A_i \oplus B_i$

Propagate becomes 1 when an incoming carry is passed to the next stage.

Carry Equations

$$C1 = G0 + P0C0$$

$$C2 = G1 + P1G0 + P1P0C0$$

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$$

All carry bits are calculated simultaneously.

Comparison

Ripple Carry Adder

Carry Look-Ahead Adder

Carry propagates sequentially Carries generated simultaneously

Slow

Fast

Delay = $O(n)$

Delay $\approx O(\log n)$

Less hardware

More hardware

Why CLA is Preferred

- Faster arithmetic operations
- Parallel carry generation
- Reduces execution time
- Used in modern high-performance processors

3. Booth's Algorithm

Multiply:

$$+11 \times (-7)$$

Binary representation:

$$+11 = 01011$$

$$-7 = 11001$$

Initial Registers:

$$A = 00000$$

$$Q = 11001$$

$$Q-1 = 0$$

$$M = 01011$$

$$-M = 10101$$

Iterations

Step Q0Q-1 Operation After Arithmetic Right Shift

1	10	$A = A - M$	Shift
2	11	No Operation	Shift
3	01	$A = A + M$	Shift
4	00	No Operation	Shift
5	10	$A = A - M$	Shift

Final Product:

1110110011

Decimal Result:

-77

Since:

$$11 \times (-7) = -77$$

Advantages of Booth's Algorithm

- Handles signed numbers directly.
- Reduces the number of additions and subtractions.
- Efficient for consecutive 1's.
- Faster multiplication in processors.

4. Restoring and Non-Restoring Division

Divide:

$$25 \div 4$$

Binary:

$$25 = 11001$$

$$4 = 00100$$

(A) Restoring Division

Initial Registers:

$$A = 00000$$

$$Q = 11001$$

$$M = 00100$$

Algorithm:

1. Shift left (A,Q).
2. Subtract divisor.
3. If remainder becomes negative:
 - Restore remainder.
 - Quotient bit = 0.
4. Otherwise:
 - Keep remainder.
 - Quotient bit = 1.

Final Result:

Quotient = 00110 = 6

Remainder = 00001 = 1

(B) Non-Restoring Division

Algorithm:

1. Shift left.
2. If previous remainder is positive, subtract divisor.
3. If previous remainder is negative, add divisor.
4. No immediate restoration is performed.

Final Result:

Quotient = 6

Remainder = 1

Comparison

Restoring Division	Non-Restoring Division
Performs restore operation	No restore operation
More arithmetic operations	Fewer arithmetic operations
Slower	Faster
Simpler hardware	Slightly complex hardware
Preferred Method	

Non-Restoring Division is preferred because it performs fewer operations, executes faster, and improves processor efficiency.

5. IEEE 754 Floating-Point Representation

Decimal Number:

+26.75

Binary Conversion:

26 = 11010

0.75 = .11

26.75 = 11010.11

Normalized Form:

1.101011×2^4

Single Precision (32-bit)

Sign Bit

0

Exponent

Bias = 127

$4 + 127 = 131$

131 = 10000011

Mantissa

101011000000000000000000

Final IEEE-754 Representation:

0 10000011 101011000000000000000000

Double Precision (64-bit)

Sign Bit

0

Exponent

Bias = 1023

$4 + 1023 = 1027$

Effect of Precision

- **Single Precision (32-bit):** Approximately 7 decimal digits of accuracy.
- **Double Precision (64-bit):** Approximately 15–16 decimal digits of accuracy with reduced rounding error.

Final Answers

1. **Addition:** $-52 + 19 = -33$ (No Overflow)
Subtraction: $-52 - 19 = -71$ (No Overflow)
2. **Carry Look-Ahead Adder:** Uses generate and propagate signals to calculate carries in parallel, making it faster than the Ripple Carry Adder.
3. **Booth's Algorithm:** $+11 \times (-7) = -77$, reducing the number of additions/subtractions while supporting signed multiplication.
4. **Division:** $25 \div 4$ gives **Quotient** = 6 and **Remainder** = 1. Non-restoring division is preferred because it is faster.
5. **IEEE 754:**
 - $26.75 = 1.101011 \times 2^4$
 - **Single Precision:** 0 10000011 101011000000000000000000
 - **Double Precision:** 0 10000000011
101011000
 - $26.75 + (-8.5) = 18.25$ after exponent alignment, mantissa subtraction, normalization, and rounding if required.